arXiv:2402.07871v1 [cs.LG] 12 Feb 2024

SCALING LAWS FOR FINE-GRAINED MIXTURE OF EXPERTS

Jakub Krajewski *
University of Warsaw
IDEAS NCBR

Jan Ludziejewski *
University of Warsaw
IDEAS NCBR

Kamil Adamczewski
IDEAS NCBR

Maciej Pióro
IPPT PAN
IDEAS NCBR

Michał Krutul
University of Warsaw
IDEAS NCBR

Szymon Antoniak
University of Warsaw
IDEAS NCBR

Kamil Ciebiera
University of Warsaw
IDEAS NCBR

Krystian Król
University of Warsaw
IDEAS NCBR

Tomasz Odrzygóźdź
TradeLink

Piotr Sankowski
University of Warsaw
IDEAS NCBR

Marek Cygan
University of Warsaw
Nomagic

Sebastian Jaszczur *
University of Warsaw
IDEAS NCBR

ABSTRACT

Mixture of Experts (MoE) models have emerged as a primary solution for reducing the computational cost of Large Language Models. In this work, we analyze their scaling properties, incorporating an expanded range of variables. Specifically, we introduce a new hyperparameter, granularity, whose adjustment enables precise control over the size of the experts. Building on this, we establish scaling laws for fine-grained MoE, taking into account the number of training tokens, model size, and granularity. Leveraging these laws, we derive the optimal training configuration for a given computational budget. Our findings not only show that MoE models consistently outperform dense Transformers but also highlight that the efficiency gap between dense and MoE models widens as we scale up the model size and training budget. Furthermore, we demonstrate that the common practice of setting the size of experts in MoE to mirror the feed-forward layer is not optimal at almost any computational budget.

1 INTRODUCTION

In recent years, we have witnessed Large Language Models (LLMs) achieve exceptional performance in tasks across numerous domains (Chowdhery et al., 2022; Yin et al., 2023; Agostinelli et al., 2023). However, training those massive models incurs high computational costs, measured in millions of GPU-hours (Touvron et al., 2023b), enabled only by enormous budgets (Scao et al., 2023) and leading to non-negligible carbon footprints (Faiz et al., 2024). To combat these obstacles, the research community has been striving to increase the efficiency of LLMs. One promising approach that has lately been gaining visibility is the use of Mixture of Experts (MoE) methods. Models such as Switch (Fedus et al., 2022) and Mixtral (Jiang et al., 2024) have already demonstrated that it is possible to achieve comparable effectiveness with significantly lower computational costs.

In the context of the current trend of increasing budgets for training language models, a question arises: will MoE models continue to be attractive in the future? This is an important issue, as other studies have stated that the gap in efficiency between MoE and standard Transformers narrows at

Contributions: Jakub implemented fine-grained MoE, ran experiments, and oversaw the course of the project. Jan designed and implemented the scaling laws, also optimized and tuned the fine-grained MoE implementation. Kamil A. provided significant advice on many aspects of the project. Maciej experimented with the block design and, with Michał, provided considerable technical support. Szymon, Kamil C., Krystian, and Tomasz contributed to the project and the engineering in various ways. Marek, along with Piotr, provided high-level scientific advice. Sebastian came up with the initial idea, started the project, and supervised it while setting the research direction and leading experiments and analyses. Correspondence to <s.jaszczur@uw.edu.pl>. * Equal contribution.

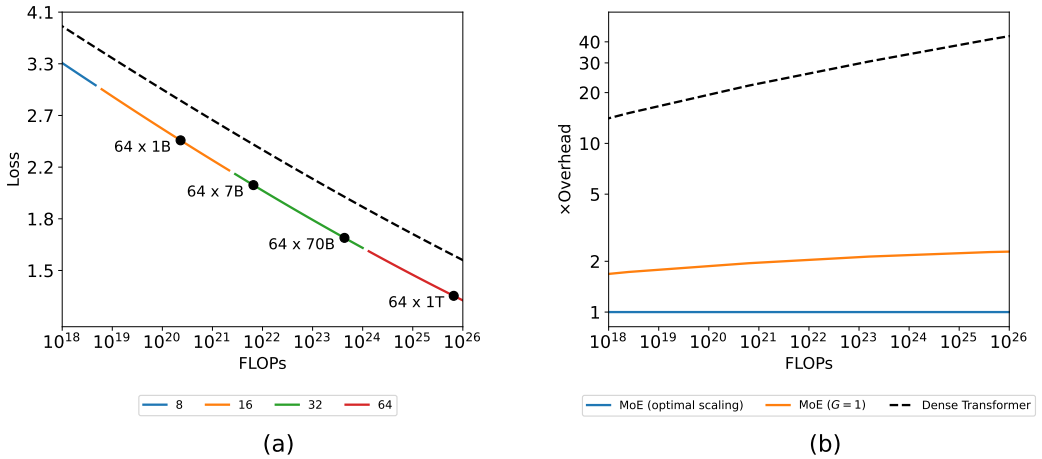

Figure 1: Mixture-of-Experts can be *always* considered more efficient than dense Transformers, regardless of the model size. **(a)** Compute Optimal scaling curves for MoE and standard Transformers. The dashed line represents a dense Transformer. Colors denote optimal granularity for the given FLOPs training budget. **(b)** Relative number of FLOPs needed to train Transformer and Vanilla MoE (MoE with $G = 1$) to achieve the performance of MoE with compute optimal G.

scale (Artetxe et al., 2022) or even that traditional dense models may outperform MoE as the size of the models increases (Clark et al., 2022).

In this paper, we argue that previous claims lose their validity when we relax certain implicit assumptions regarding the training process, present in previous research. In particular, we refer to the fixed training duration and the constant size of experts in MoE models.

Our results suggest that a compute-optimal MoE model trained with a budget of 10^{20} FLOPs will achieve the same quality as a dense Transformer trained with a $20\times$ greater computing budget, with the compute savings rising steadily, exceeding $40\times$ when budget of 10^{25} FLOPs is surpassed (see Figure 1). Importantly, we show that the standard practice of fixing the size of experts in MoE to be the same as feed-forward layer is *almost never* optimal.

Our main contributions are:

1. Introducing a new hyperparameter - granularity. Adjusting this parameter allows us to determine the optimal size of experts in MoE models, which translates into increased efficiency.
2. Deriving new scaling laws for MoE models that incorporate variable training duration, the number of parameters, and granularity. Such scaling laws allow us to calculate optimal training hyperparameters for MoE models.
3. Demonstrating that, with optimal settings, MoE models can always outperform traditional Transformers at any computing budget. This is a conclusion contrary to the results from Clark et al. (2022).

The code used to produce the results described in this work is open-sourced at `github.com/llm-random/llm-random`.

2 Related Work

Mixture of Experts. In the context of language modeling, MoE was first introduced by Shazeer et al. (2017) as a sparsely gated layer between stacked blocks of LSTM (Hochreiter & Schmidhuber, 1997). A similar technique was proposed in the context of Transformers by Shazeer et al. (2018) and Lepikhin et al. (2020). Fedus et al. (2022) proposed to route each input to only a single expert and designed a modified initialization scheme to reduce training instability. Numerous studies

have proposed to modify the original routing method. Lewis et al. (2021) used a linear assignment algorithm to postprocess token-expert mappings and ensure even expert selections. Roller et al. (2021) suggested another approach involving deterministic hash functions. Zhou et al. (2022) proposed expert choice routing, eliminating the need for additional load balancing losses. Puigcerver et al. (2023) designed a fully-differentiable Soft MoE architecture.

Concurrently to our work, Dai et al. (2024) proposed to modify the MoE layer by segmenting experts into smaller ones and adding shared experts to the architecture. Independently, Liu et al. (2023) suggested a unified view of sparse feed-forward layers, considering, in particular, varying the size of memory blocks. Both approaches can be interpreted as modifying granularity. However, we offer a comprehensive comparison of the relationship between training hyperparameters and derive principled selection criteria, which they lack.

Scaling laws. Scaling laws are empirically derived equations relating the loss of a model with variables such as the number of parameters, training samples, or the computational budget. In the case of dense Transformers, scaling laws were first studied by Kaplan et al. (2020), who observed power law relationships between the final model perplexity and model and dataset size. This work was extended by Hoffmann et al. (2022) by considering variable cosine cycle lengths and formulating a modified functional form of the scaling equation.

Scaling laws have also been proposed for other architectures and training scenarios. Henighan et al. (2020) studied autoregressive modeling across various modalities, while Ghorbani et al. (2021) considered machine translation. Frantar et al. (2023) explored the impact of pruning on vision and language Transformers, deriving optimal sparsity for a given compute budget. Clark et al. (2022) studied the scaling of MoE when changing model size and number of experts on a fixed dataset, concluding that routed models are more efficient only until a certain model size. In this work, we challenge that claim by considering a variable, optimal dataset size for both model families (see Section 6.3).

3 BACKGROUND

3.1 MODEL ARCHITECTURE

Transformer. A standard decoder-only Transformer (Radford et al., 2018a;b; Kaplan et al., 2020; Brown et al., 2020) consists of an embedding layer, a stack of alternating attention and feed-forward layers, and an unembedding layer. In the model, each input token is converted by the embedding layer into a vector of size d_{model}, the dimension maintained across all the layers in the residual stream.

The feed-forward component consists of two linear transformations and a nonlinearity ϕ in between. It can be described as $\text{FFN}(x) = \phi(xW_1 + b_1)W_2 + b_2$, with W_1 mapping from d_{model} to d_{ff}, and W_2 back to the original d_{model}. It is standard (Radford et al., 2018a; Rae et al., 2022; Touvron et al., 2023a; Jiang et al., 2023) to set the hidden dimension as $d_{\text{ff}} = 4 \cdot d_{\text{model}}$.

Feed-forward layers contain the majority of Transformer parameters and require the biggest computational budget counted in terms of FLOPs. Subsequently, they are the main focus of the Mixture of Experts models considered in this work.

Mixture of Experts. The core idea behind MoE in Transformers is to replace the feed-forward layer with a set of N_{expert} *experts*. The size of each expert is typically (Fedus et al., 2022; Zhou et al., 2022; 2023; Jiang et al., 2024) set to mirror the original dimensions of the layer, with the hidden expert dimension d_{expert} equal to d_{ff}. Therefore, the total number of parameters in MoE scales linearly with the number of experts. However, the computational cost remains approximately constant as each input is routed and then processed by a subset of experts.

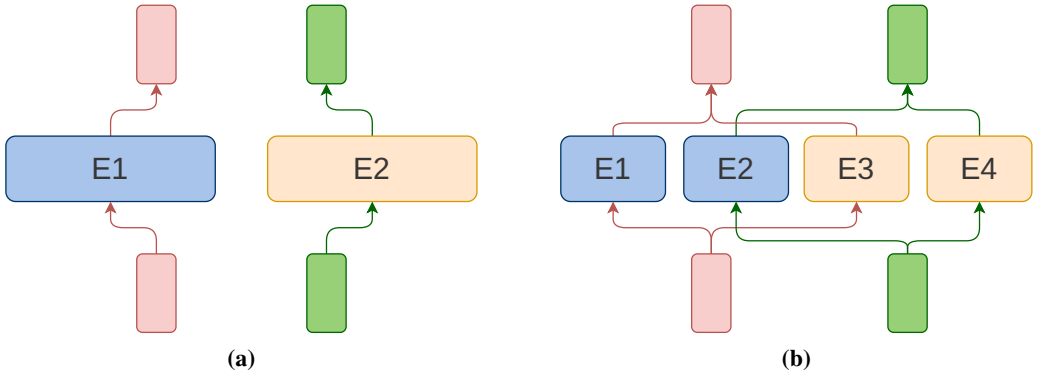

Figure 2: **(a)** Standard MoE layer with $G = 1$ **(b)** Corresponding MoE layer with $G = 2$. Each of the original experts is split into two granular ones. The split occurs in the hidden dimension of an expert. Increasing G allows for a more precise mapping between experts and tokens. Since for granularity G, the token is routed to G granular experts, the number of parameters activated per token is the same in both cases.

3.2 Scaling Laws

Dense Transformers. Large Transformer-based models are known to approximately obey the power-law relationship between final loss $\mathcal{L}$, model size N, and number of training tokens D. This relationship is often called *Chinchilla scaling laws* described by Hoffmann et al. (2022) as

$$\mathcal{L}(N, D) = c + \frac{a}{N^{\alpha}} + \frac{b}{D^{\beta}}. \tag{1}$$

The power-law formula is composed of three distinct terms that characterize the intrinsic entropy of data, constraints of the model, and limitations in the training data. The term c represents the minimum possible error intrinsic to the data. The remaining two terms are suboptimality terms, which address the limitations in function representation owing to the size of the model and in data signified by the number of tokens. In the limit, with infinite data and model size, the loss is reduced to c.

Mixture of Experts. For MoE Transformer-based models, Clark et al. (2022) formulated the final loss for a constant dataset size D of 130B tokens, allowing for variations in the expansion rate E, as:

$$\mathcal{L}(N, E) = \left(\frac{10^{d/a}}{N}\right)^{a} \left(\frac{1}{E}\right)^{b + c \log N}. \tag{2}$$

However, this result has a notable limitation as it can be applied only to the original dataset size. The scalability and effectiveness are constrained in this scenario because it is crucial to align the number of training samples with the available computational resources for optimal use. As per Kaplan et al. (2020) and Hoffmann et al. (2022), maintaining a constant dataset size while scaling up the neural network size leads to undertraining, resulting in a model that does not perform to its full potential.

4 Granularity

As described in Section 3, in the standard setting, the inner dimension of each expert network, d_{expert}, is equal to d_{ff}, which is the same size as the feed-forward layer of the base model.

In this work, we suggest an alternative approach where the hidden dimension of the expert is not necessarily set to mirror that of the standard feed-forward layer. Instead, it can be adjusted to a value that is the most effective. This approach allows the configuration of MoE to be articulated in terms of two key hyperparameters: *granularity* (G) and *expansion rate* (E). In the following parts of this work, we will also use the term *active* parameters to refer to the non-embedding parameters used to produce output for a single token, except routing. The number of active parameters is denoted as N_{act}.

Let d_{expert} be the hidden dimension of a single expert. Granularity is defined as

$$G = \frac{d_{\text{ff}}}{d_{\text{expert}}}.$$

In other words, granularity denotes the multiplier factor for the change in the size of an expert from the original standard model, defined as $G = 1$. In this work, we investigate $G > 1$ where experts are smaller than in the standard layer.

Note that increasing granularity does not affect the number of active parameters. As G increases, the number of experts that process the token grows proportionally to G. In other words, for granularity G, a token is routed to G fine-grained experts, thereby keeping the number of active parameters constant. See Fig. 2 for visualization.

We then define the *expansion rate*, which describes the increase in the number of parameters from a standard transformer layer to a MoE layer. Given that, N_{MoE} and N_{ff} denote the total number of parameters in a MoE layer excluding routing and the standard feed-forward layer, respectively. The expansion rate E is then defined as

$$E = \frac{N_{\text{MoE}}}{N_{\text{ff}}}.$$

Expansion rate can also be seen as the total number of parameters in a MoE layer compared to its active parameters.

The concept of the expansion rate is intricately linked to the number of experts through the idea of granularity. Indeed, the definitions of both granularity and expansion rate extend and refine our understanding of the number of experts, symbolized as N_{expert}.

$$N_{\text{expert}} = G \cdot E \tag{3}$$

For non-granular models, where $G = 1$, the expansion rate is equal to the number of experts.

Intuitively, increasing granularity for a given expansion rate gives the model more flexibility in mapping datapoints to experts, potentially improving performance. We incorporate the notion of granularity into our scaling laws in Section 5. The discussion about practical tradeoffs in changing this parameter is given in Section 6.

5 Scaling Laws

Granularity determines changes in the architecture of MoE. In this section, we answer a central question of this work: whether the granular MoE models follow scaling laws and, if so, how granularity affects them. Thus, we aim to derive a parametric scaling law for predicting the final loss value $\mathcal{L}$ based on granularity G, total number of non-embedding parameters N, and number of training tokens D.

We run over 100 experiments on the decoder-only Transformer architecture, with each feed-forward component replaced by a Mixture of Experts layer. Those experiments involve training models with sizes ranging from 129M to 3.7B parameters across different training durations, from 16B to 130B tokens. We consider logarithmically spaced values of granularity between 1 and 16. To constrain the search space, $E = 64$ is fixed, following the recommendations of Clark et al. (2022). In addition, we also run experiments with dense Transformers to compare their performance with MoE. The details of all architectures, the training procedure, and hyperparameter choices are described in detail in Appendix A.

In the subsequent part of this paper, we will use the notation $E \times N_{\text{act}}$ to describe a MoE model with N_{act} active parameters and expansion rate E.

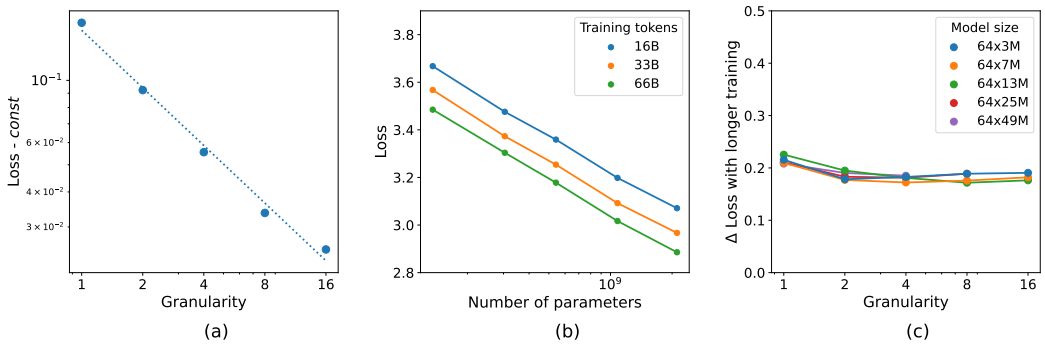

Figure 3: **(a)** The effect of G on $\mathcal{L}_{N,D}(G)$ for constant N and D. Both axes are in the log-scale. The results suggest the linear relationship between $\log(G)$ and $\log(\mathcal{L} - c)$. The given values are $N = 64 \times 25M$, $D = 16B$, $const = 3.12$. The plots for additional values of N and D can be found in Appendix F. **(b)** The impact of varying the number of parameters N on the loss for fixed granularity $G = 4$. For other granularity values, see Appendix F. **(c)** The difference in the loss between training for 16B and 65B tokens for all model sizes and granularity values. The model size is reported as the expansion rate and the number of active parameters.

5.1 Power Law With Respect to Granularity

We first answer the question of whether granular models follow the scaling laws. In Figure 4(a), it can be seen that increasing granularity results in a lower loss. The returns follow approximately an exponential pattern, converging to a positive constant. The empirical relationship given by Figure 3(a) suggests the following power-law dependence of loss on a varying granularity for given N and D and constants g, h and γ that may be dependent on them,

$$\mathcal{L}_{N,D}(G) = \frac{g_{N,D}}{G^{\gamma_{N,D}}} + h_{N,D}. \tag{4}$$

5.2 Scaling the Model and Dataset Size

As outlined in Section 3.2, the power-law given by Eq. 1 consists of three terms that describe inherent data entropy and limitations in function representation and data. This derivation is independent of the architecture. In particular, the Eq. 1 also holds for constant granularity. Empirically, we observe a power law relationship in N and D analogous to that in dense models as depicted in Figure 3(b) for a fixed value of granularity (see also Fig. 1, Kaplan et al. (2020)). Furthermore, the validity of this functional form is verified by fit in Section 5.4.

Since we know that separate scaling laws are valid for given granularities, in the general form, the parameters in Eq. 1 can be dependent on the model's granularity:

$$\mathcal{L}_G(N, D) = c_G + \frac{a_G}{N^{\alpha_G}} + \frac{b_G}{D^{\beta_G}}. \tag{5}$$

5.3 The Form of the Joint Scaling Law

Following the above observation that models with constant granularity obey Chinchilla scaling laws given by Eq. 1, the key question arises as to how the general notion of granularity G can be incorporated into the joint scaling law. Moreover, the scaling law formula from Eq. 5 for constant N and D has to be representable by Eq. 4. This is because the former is a more general equation, encompassing shared hyper-parameters across all N, D, and G. It is anticipated to align with the latter, consisting of distinct power laws, each with specific parameters for different N and D values. Consequently, the objective is to identify a function that fulfills these criteria.

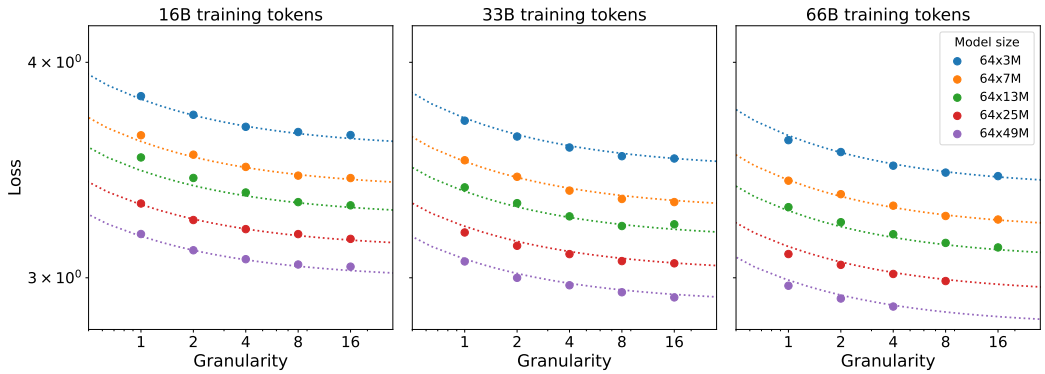

Figure 4: Fit of the scaling laws compared to the experimental results.

$$\mathcal{L}(N, D, G) = \mathcal{L}_{N,D}(G) = \mathcal{L}_G(N, D) \tag{6}$$
$$= \frac{g_{N,D}}{G^{\gamma_{N,D}}} + h_{N,D} = c_G + \frac{a_G}{N^{\alpha_G}} + \frac{b_G}{D^{\beta_G}}$$

In the subsequent sections, we aim to determine which of these parameters remain independent of G and identify their functional form. Furthermore, we present some rationale for the structure of our formula.

Lower Bound. Consider the limit of Eq. 5 for N and D growing to infinity:

$$\lim_{\substack{N\to\infty \\ D\to\infty}} \mathcal{L}(N, D, G) = c_G. \tag{7}$$

with the constant term c_G dependent on granularity. This is contradictory to the fact that it captures the inherent entropy of the dataset. Lower bound of the achievable loss when training bigger models on more samples should not depend on the architecture, therefore parameter $c_G = c$ is constant for all granularities.

Granularity and Number of Tokens D. As seen in Figure 3(c), the benefit of training a model on a larger dataset is almost the same for each granularity value. This suggests that there is no interaction between D and G. Therefore, we can assume that

$$\frac{b_G}{D^{\beta_G}} = \frac{b}{D^{\beta}}. \tag{8}$$

Granularity and Model Size N. We consider α to be a constant that describes how the function scales with N. In this work, we assume polynomial functional forms that rule out the potential dependency of α on G given the form of Eq. 4. Therefore, the only element dependent on G is a_G:

$$\mathcal{L}(N, D, G) = c + \left(\frac{g}{G^{\gamma}} + a\right)\frac{1}{N^{\alpha}} + \frac{b}{D^{\beta}}. \tag{9}$$

Finally, one could consider omitting the constant a in the equation above, and it would still reduce to 4 for constant N and D. However, this would mean that a model with infinite granularity and a small number of active parameters can achieve the perfect perplexity of the lower bound. We assume that a sparse MoE (Mixture of Experts) model is unlikely to surpass the performance of an equivalent dense model that has a matching total number of parameters, all of which are active. This means that constant a can act as a marginal improvement due to granularity.

Subsequently, we fit parameters in Eq. 9 to describe the scaling of MoE. For comparison, we also perform fitting for dense transformer given by Eq. 1. Similarly to Hoffmann et al. (2022), we use Huber loss (Huber, 1964), with $\delta = 0.1$. The optimization is performed using the BFGS algorithm. We include a weight decay of $5e-4$ to enhance generalization. We start with fitting parameters in Eq. 9 and then find architecture-dependent coefficients α, β, A and B in Eq. 1. We observe a good fit, with RMSE $= 0.015$. The values are presented in Table 1. We depict the results in Figure 4.

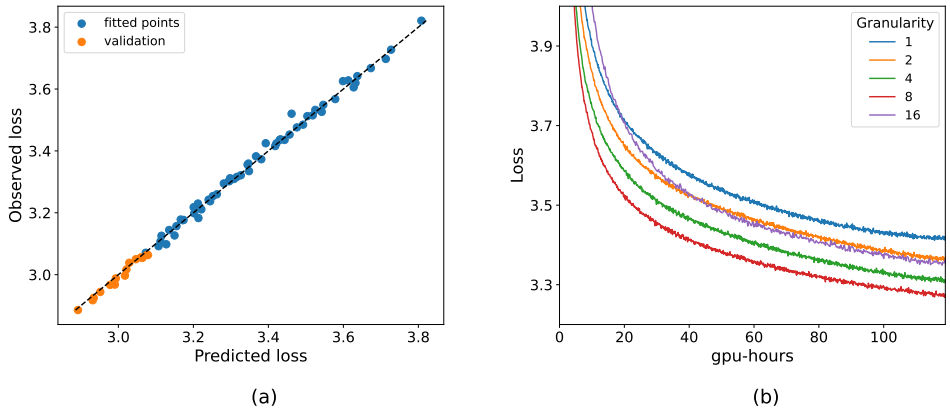

Figure 5: **(a)** Validation of the scaling laws. **(b)** Training loss curves for model with $N = 64 \times 7M$, $D = 66B$ tokens, measured against wall-clock time on NVIDIA A100 GPU. $G = 8$ leads to the best performance, as for $G = 16$ the routing cost dominates gains from granularity. We model the increased cost of routing by measuring FLOPs for each configuration.

5.4 Fitting the Parametric Scaling Law

Table 1: Values of the fitted coefficients.

Model	a	α	b	β	g	γ	c
MoE	18.1	0.115	30.8	0.147	2.1	0.58	0.47
Dense	16.3	0.126	26.7	0.127	-	-	0.47

We validate the stability of the fit by excluding the top 20% of models with the lowest perplexity and finding the coefficients based on the remaining experiments. We observe that the formula remains almost unchanged in this scenario (see Table 5 in Appendix B). The validation RMSE is 0.019. Results are depicted in Figure 5 (a).

5.5 MoE Scaling Properties

Comparing the part of the formula that approximates underfitting (that is, dependent on training tokens) in MoE ($30.8D^{-0.147}$) and Transformer ($26.7D^{-0.127}$), we can infer that MoE models need longer training to perform competitively but scale better after reaching that point. Nonetheless, this moment may still precede the compute optimal for both models. On the other hand, we can see that the exponent on dense models $\alpha = -0.126$ scales better with a total number of parameters than the MoE counterpart $\alpha = -0.115$. This should not be surprising since dense models use all parameters on each token contrary to MoE, which gains a computational advantage by activating only a subset of them. Therefore, the fair comparison of the performance has to take into account FLOPs used by each model type. In the next section, we find compute-optimal granularity for a given FLOP budget.

6 Optimal Allocation of Computational Budget

In Section 5, we show that higher granularity leads to lower loss for the same number of training steps. This is not always the case if we consider the wall-clock time. As depicted in Figure 5 (b), in practice for too high values of G (relative to d_{model}), training can be bottlenecked by the routing cost. Practical modeling of this situation is possible by measuring FLOPs in routing. In this section we find optimal N, D, G for a given computational budget F by solving the following optimization problem,

$$\begin{aligned} \underset{N,D,G}{\text{minimize}} \quad & \mathcal{L}(N, D, G) \\ \text{subject to} \quad & \text{FLOPs}(N, D, G) = F. \end{aligned}$$

6.1 Computational Cost of Granularity

It is important to acknowledge that increasing granularity can lead to some challenges in training the model, namely higher computational and communication costs and a larger memory footprint. The main component responsible for higher costs is the increase in routing operations due to a larger pool of granular experts. This increase is proportional to the value of G. For standard, non-granular MoE models ($G = 1$), the routing overhead still exists, although it has been considered negligible.

Taking into account the routing operation overhead, the number of used FLOPs F is described by the following formula:

$$F = (12{d_{\text{model}}}^2 c_f + d_{\text{model}} E G c_r) \cdot D \cdot n_{\text{blocks}}, \tag{10}$$

given expansion rate E, granularity G, and constants that denote FLOPs per active parameter ratio, respectively, within routing (c_r) and within the rest of the network (c_f). The term $12{d_{\text{model}}}^2$ is the number of active parameters within a transformer block, while $d_{\text{model}} E G c_r$ is the number of active parameters within a routing network. The in-depth analysis of constants c_r and c_f can be found in Appendix E. We exclude embedding and unembedding from the FLOPs calculations, following Hoffmann et al. (2022).

Observe that, in contrast to scenarios where routing operations are omitted, the FLOPs calculation that incorporates routing overhead relies on both d_{model} and n_{blocks}. Consequently, an additional condition is required to determine the scaling of d_{model} and n_{blocks} in relation to an increase in N, the number of parameters. It is noted that minor variations in the depth-to-width ratio are not significant (Kaplan et al., 2020). Following this analysis, we opt to adopt the assumption that $d_{\text{model}} = 64 n_{\text{blocks}}$.

The total number of parameters in the feed-forward layer, excluding the routing matrix, is $2E d_{\text{ff}} d_{\text{model}} = 8E{d_{\text{model}}}^2$, and $4{d_{\text{model}}}^2$ in attention (key, query, value, and output projection). This results in the following formula for the total number of parameters, $N = {d_{\text{model}}}^2 \cdot (8E + 4) \cdot n_{\text{blocks}}$.

6.2 Compute Optimal Formula

Taking into consideration we need to solve the following optimization problem, given F,

$$\begin{aligned} \underset{N,D,G}{\text{minimize}} \quad & \mathcal{L}(N, D, G) \\ \text{subject to} \quad & F = (12{d_{\text{model}}}^2 c_f + d_{\text{model}} E G c_r) \cdot D \cdot n_{\text{blocks}} \\ & N = d_{\text{model}}^2 \cdot (8E + 4) \cdot n_{\text{layers}}, \\ & d_{\text{model}} = 64 \cdot n_{\text{layers}}. \end{aligned}$$

All these constraints are reducible to a one-dimensional optimization problem, which is, however, hard to solve analytically. Therefore we approximate the solution using Brent's method (Brent, 1971). The results of this optimization for varying FLOPs budgets are plotted in Figure 1 while the optimal configurations of parameters for selected model sizes are presented in Table 2. To validate the uncertainty of these predictions, we follow Hoffmann et al. (2022) and calculate the 10th and 90th percentiles estimated via bootstrapping data (see Appendix C for the detailed results).

6.3 MoE is Always More Efficient

Contrary to the results from Clark et al. (2022), in Figure 1 we can see, that Mixture-of-Experts can be always considered more efficient than dense Transformers, regardless of the model size. According to our previous observations from Section 5.5, MoE models scale better with optimal training. However, for short training schedules, they may under-perform dense models. This means that for constant training time and increasing model size, there exists a point where both models will become very under-trained, in which scenario dense models surpass MoE. This shows why in Clark et al. (2022), where varying the number of training tokens has not been considered, MoE was predicted to be under-performing for models bigger than $1T$. However, when all training hyper-parameters N, D, G are properly selected to be compute-optimal for each model, the gap between dense and sparse models only increases as we scale.

Table 2: Compute optimal training hyper-parameters for MoE models. Optimal N and D follow approximately similar relation to these of Hoffmann et al. (2022) for active parameters around the range of $1B$ to $10B$ parameters, requiring comparably longer training for smaller models and shorter for bigger ones. Higher granularity is optimal for larger compute budgets.

N	D	G	FLOPs	Loss
64 x 100M	4.37B	8	2.95e+18	3.133
64 x 1B	28.94B	16	1.93e+20	2.491
64 x 3B	72.90B	16	1.41e+21	2.245
64 x 7B	137.60B	32	6.46e+21	2.076
64 x 70B	941.07B	32	4.16e+23	1.694
64 x 300B	2.96T	64	5.69e+24	1.503
64 x 1T	7.94T	64	4.97e+25	1.367

7 Discussion

Extreme Granularity. In Section 5, we argue that model performance improves with increasing granularity. This postulate largely aligns with the empirical findings of our study. Nonetheless, at exceedingly high granularity levels, such as $G = 64$ in models characterized by $d_{\text{model}} = 256$ and $E = 64$, there is an observable decline in performance. This phenomenon is particularly evident in scenarios where the number of parameters in the routing mechanism exceeds active parameters in actual experts. Additionally, as described in Section 6, the utility of such high granularity is predominantly restricted to models of substantial size. In alignment with the principles outlined by Hoffmann et al. (2022), this research focuses more on findings that can be broadly applied rather than delving into the specific details of these corner-case situations. However, it is hypothesized that the efficiency of models with significantly high granularity could be potentially enhanced through careful expert initialization or modifications to the routing algorithm. These ideas are set aside to be investigated in future studies.

Varying Expansion Rate. In this study, due to computational resources constraint, we focus on $E = 64$, as recommended by Clark et al. (2022). This value of E was also used for the largest models in other works (Du et al., 2022; Zhou et al., 2022) and the best-performing configuration in Fedus et al. (2022). Nonetheless, we acknowledge the importance of considering different expansion rates, as different levels of E may be chosen based on factors like the target size of the model in memory. Therefore, in Appendix D, we present the results of the study for $E = 16$ and show that the main findings of this work are still valid in such cases.

Including E in the formula. Another possible advancement would be to unify all of the factors N, D, G and E in one formula. While this would open the possibility of studying the relationships between coefficients in more detail, it would also be hard to practically recommend the optimal configuration in such a scenario using only FLOPs. This is because larger values of E typically lead to better performance but also incur additional memory requirements. Therefore, the choice of expansion rate may be heavily dependent on the available hardware configuration. We leave a detailed study of these factors for future work.

Modeling the cost of granularity. It is important to note that the exact estimation of the training cost of MoE models is dependent on the training setup, hardware, and implementation. Specifically, increasing G can lead to higher transfer costs, depending on the adopted model of distributed training. Therefore, the precise selection of hyperparameters should be made considering these factors. In this work, we model the cost of operations using FLOPs, which is common in the Scaling Laws literature (Kaplan et al., 2020; Hoffmann et al., 2022; Frantar et al., 2023). Additionally, we would like to note that in our setup, we observe significant gains of fine-grained MoE measured as wall-clock time needed to achieve given perplexity (see Fig. 5 (b) for an example).

8 CONCLUSIONS

This study introduces a novel hyperparameter, granularity (G), and underscores the significance of adjusting it for optimizing the efficiency of experts within MoE models. A central finding of this research is that a standard granularity of $G = 1$ is suboptimal across a broad range of FLOPs, leading to the recommendation of using higher granularity values to enhance MoE model performance and efficiency. Simultaneously, this work emphasizes the importance of varying training duration for compute-optimal settings. Consequently, both granularity and variable training length are incorporated into new scaling laws. These laws confidently demonstrate that MoE models consistently outperform dense transformers in terms of efficiency and scaling. This work not only sheds new light on the scaling laws applicable to MoE models but also provides practical guidance for improving computational efficiency in large language models. The insights are critical for the development and optimization of large-scale language models, marking a significant advancement in the field.

9 REPRODUCIBILITY

The code used to produce the results described in this work is open-sourced and can be found at `github.com/llm-random/llm-random`.

ACKNOWLEDGMENTS

We would like to express sincere gratitude to Piotr Miłoś and Tomasz Trzciński for valuable feedback and to Aleksandra Weglarz for her help with graphic design.

This work was funded by IDEAS NCBR, which also provided significant computational resources a supportive research environment and direction. The research was supported by PL-Grid infrastructure (grant PLG/2023/016148). We also benefited from the Entropy cluster (hosted at the Faculty of Mathematics, Informatics and Mechanics of the University of Warsaw) funded by NVIDIA, Intel, the Polish National Science Center grant 2022/45/N/ST6/02222, and ERC Starting Grant TOTAL. Marek Cygan was partially supported by an NCBiR grant POIR.01.01.01-00-0392/17-00.

REFERENCES

Andrea Agostinelli, Timo I. Denk, Zalán Borsos, Jesse Engel, Mauro Verzetti, Antoine Caillon, Qingqing Huang, Aren Jansen, Adam Roberts, Marco Tagliasacchi, Matt Sharifi, Neil Zeghidour, and Christian Frank. Musiclm: Generating music from text, 2023.

Mikel Artetxe, Shruti Bhosale, Naman Goyal, Todor Mihaylov, Myle Ott, Sam Shleifer, Xi Victoria Lin, Jingfei Du, Srinivasan Iyer, Ramakanth Pasunuru, Giri Anantharaman, Xian Li, Shuohui Chen, Halil Akin, Mandeep Baines, Louis Martin, Xing Zhou, Punit Singh Koura, Brian O'Horo, Jeff Wang, Luke Zettlemoyer, Mona Diab, Zornitsa Kozareva, and Ves Stoyanov. Efficient large scale language modeling with mixtures of experts, 2022.

Richard P. Brent. An algorithm with guaranteed convergence for finding a zero of a function. *Comput. J.*, 14:422–425, 1971. URL `https://api.semanticscholar.org/CorpusID:10312755`.

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners, 2020.

Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayana Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. Palm: Scaling language modeling with pathways, 2022.

Aidan Clark, Diego de las Casas, Aurelia Guy, Arthur Mensch, Michela Paganini, Jordan Hoffmann, Bogdan Damoc, Blake Hechtman, Trevor Cai, Sebastian Borgeaud, George van den Driessche, Eliza Rutherford, Tom Hennigan, Matthew Johnson, Katie Millican, Albin Cassirer, Chris Jones, Elena Buchatskaya, David Budden, Laurent Sifre, Simon Osindero, Oriol Vinyals, Jack Rae, Erich Elsen, Koray Kavukcuoglu, and Karen Simonyan. Unified scaling laws for routed language models, 2022.

Damai Dai, Chengqi Deng, Chenggang Zhao, R. X. Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Y. Wu, Zhenda Xie, Y. K. Li, Panpan Huang, Fuli Luo, Chong Ruan, Zhifang Sui, and Wenfeng Liang. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models, 2024.

Nan Du, Yanping Huang, Andrew M. Dai, Simon Tong, Dmitry Lepikhin, Yuanzhong Xu, Maxim Krikun, Yanqi Zhou, Adams Wei Yu, Orhan Firat, Barret Zoph, Liam Fedus, Maarten Bosma, Zongwei Zhou, Tao Wang, Yu Emma Wang, Kellie Webster, Marie Pellat, Kevin Robinson, Kathleen Meier-Hellstern, Toju Duke, Lucas Dixon, Kun Zhang, Quoc V Le, Yonghui Wu, Zhifeng Chen, and Claire Cui. Glam: Efficient scaling of language models with mixture-of-experts, 2022.

Ahmad Faiz, Sotaro Kaneda, Ruhan Wang, Rita Osi, Prateek Sharma, Fan Chen, and Lei Jiang. Llmcarbon: Modeling the end-to-end carbon footprint of large language models, 2024.

William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity, 2022.

Elias Frantar, Carlos Riquelme, Neil Houlsby, Dan Alistarh, and Utku Evci. Scaling laws for sparsely-connected foundation models, 2023.

Behrooz Ghorbani, Orhan Firat, Markus Freitag, Ankur Bapna, Maxim Krikun, Xavier Garcia, Ciprian Chelba, and Colin Cherry. Scaling laws for neural machine translation, 2021.

Tom Henighan, Jared Kaplan, Mor Katz, Mark Chen, Christopher Hesse, Jacob Jackson, Heewoo Jun, Tom B. Brown, Prafulla Dhariwal, Scott Gray, Chris Hallacy, Benjamin Mann, Alec Radford, Aditya Ramesh, Nick Ryder, Daniel M. Ziegler, John Schulman, Dario Amodei, and Sam McCandlish. Scaling laws for autoregressive generative modeling, 2020.

Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8): 1735–1780, 1997.

Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. Training compute-optimal large language models, 2022.

Peter J. Huber. Robust Estimation of a Location Parameter. *The Annals of Mathematical Statistics*, 35(1):73 – 101, 1964. doi: 10.1214/aoms/1177703732. URL `https://doi.org/10.1214/aoms/1177703732`.

Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Lélio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. Mistral 7b, 2023.

Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, Gianna Lengyel, Guillaume Bour, Guillaume Lample, Lélio Renard Lavaud, Lucile Saulnier, Marie-Anne Lachaux, Pierre Stock, Sandeep Subramanian, Sophia Yang, Szymon Antoniak, Teven Le Scao, Théophile Gervet, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. Mixtral of experts, 2024.

Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models, 2020.

Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. Gshard: Scaling giant models with conditional computation and automatic sharding, 2020.

Mike Lewis, Shruti Bhosale, Tim Dettmers, Naman Goyal, and Luke Zettlemoyer. Base layers: Simplifying training of large, sparse models, 2021.

Zeyu Leo Liu, Tim Dettmers, Xi Victoria Lin, Veselin Stoyanov, and Xian Li. Towards a unified view of sparse feed-forward network in pretraining large language model, 2023.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization, 2019.

Joan Puigcerver, Carlos Riquelme, Basil Mustafa, and Neil Houlsby. From sparse to soft mixtures of experts, 2023.

Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. Improving language understanding by generative pre-training. 2018a.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. 2018b. URL `https://d4mucfpksywv.cloudfront.net/better-language-models/language-models.pdf`.

Jack W. Rae, Sebastian Borgeaud, Trevor Cai, Katie Millican, Jordan Hoffmann, Francis Song, John Aslanides, Sarah Henderson, Roman Ring, Susannah Young, Eliza Rutherford, Tom Hennigan, Jacob Menick, Albin Cassirer, Richard Powell, George van den Driessche, Lisa Anne Hendricks, Maribeth Rauh, Po-Sen Huang, Amelia Glaese, Johannes Welbl, Sumanth Dathathri, Saffron Huang, Jonathan Uesato, John Mellor, Irina Higgins, Antonia Creswell, Nat McAleese, Amy Wu, Erich Elsen, Siddhant Jayakumar, Elena Buchatskaya, David Budden, Esme Sutherland, Karen Simonyan, Michela Paganini, Laurent Sifre, Lena Martens, Xiang Lorraine Li, Adhiguna Kuncoro, Aida Nematzadeh, Elena Gribovskaya, Domenic Donato, Angeliki Lazaridou, Arthur Mensch, Jean-Baptiste Lespiau, Maria Tsimpoukelli, Nikolai Grigorev, Doug Fritz, Thibault Sottiaux, Mantas Pajarskas, Toby Pohlen, Zhitao Gong, Daniel Toyama, Cyprien de Masson d'Autume, Yujia Li, Tayfun Terzi, Vladimir Mikulik, Igor Babuschkin, Aidan Clark, Diego de Las Casas, Aurelia Guy, Chris Jones, James Bradbury, Matthew Johnson, Blake Hechtman, Laura Weidinger, Iason Gabriel, William Isaac, Ed Lockhart, Simon Osindero, Laura Rimell, Chris Dyer, Oriol Vinyals, Kareem Ayoub, Jeff Stanway, Lorrayne Bennett, Demis Hassabis, Koray Kavukcuoglu, and Geoffrey Irving. Scaling language models: Methods, analysis & insights from training gopher, 2022.

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer, 2023.

Stephen Roller, Sainbayar Sukhbaatar, Arthur Szlam, and Jason Weston. Hash layers for large sparse models, 2021.

Teven Le Scao, Angela Fan, Christopher Akiki, Ellie Pavlick, Suzana Ilić, Daniel Hesslow, Roman Castagné, Alexandra Sasha Luccioni, François Yvon, Matthias Gallé, Jonathan Tow, Alexander M. Rush, Stella Biderman, Albert Webson, Pawan Sasanka Ammanamanchi, Thomas Wang, Benoît Sagot, Niklas Muennighoff, Albert Villanova del Moral, Olatunji Ruwase, Rachel Bawden, Stas Bekman, Angelina McMillan-Major, Iz Beltagy, Huu Nguyen, Lucile Saulnier, Samson Tan, Pedro Ortiz Suarez, Victor Sanh, Hugo Laurençon, Yacine Jernite, Julien Launay, Margaret Mitchell, Colin Raffel, Aaron Gokaslan, Adi Simhi, Aitor Soroa, Alham Fikri Aji, Amit Alfassy, Anna Rogers, Ariel Kreisberg Nitzav, Canwen Xu, Chenghao Mou, Chris Emezue, Christopher Klamm, Colin Leong, Daniel van Strien, David Ifeoluwa Adelani, Dragomir Radev, Eduardo González Ponferrada, Efrat Levkovizh, Ethan Kim, Eyal Bar Natan, Francesco De Toni, Gérard Dupont, Germán Kruszewski, Giada Pistilli, Hady Elsahar, Hamza Benyamina, Hieu Tran, Ian Yu, Idris Abdulmumin, Isaac Johnson, Itziar Gonzalez-Dios, Javier de la Rosa, Jenny Chim, Jesse Dodge, Jian Zhu, Jonathan Chang, Jörg Frohberg, Joseph Tobing, Joydeep Bhattacharjee, Khalid Almubarak, Kimbo Chen, Kyle Lo, Leandro Von Werra, Leon Weber, Long Phan, Loubna Ben allal, Ludovic Tanguy, Manan Dey, Manuel Romero Muñoz, Maraim Masoud, María Grandury, Mario Šaško, Max Huang, Maximin Coavoux, Mayank Singh, Mike Tian-Jian Jiang, Minh Chien Vu, Mohammad A. Jauhar, Mustafa Ghaleb, Nishant Subramani, Nora Kassner, Nurulaqilla Khamis, Olivier Nguyen, Omar Espejel, Ona de Gibert, Paulo Villegas, Peter Henderson, Pierre Colombo, Priscilla Amuok, Quentin Lhoest, Rheza Harliman, Rishi Bommasani, Roberto Luis López, Rui Ribeiro, Salomey Osei, Sampo Pyysalo, Sebastian Nagel, Shamik Bose, Shamsuddeen Hassan Muhammad, Shanya Sharma, Shayne Longpre, Somaieh Nikpoor, Stanislav Silberberg, Suhas Pai, Sydney Zink, Tiago Timponi Torrent, Timo Schick, Tristan Thrush, Valentin Danchev, Vassilina Nikoulina, Veronika Laippala, Violette Lepercq, Vrinda Prabhu, Zaid Alyafeai, Zeerak Talat, Arun Raja, Benjamin Heinzerling, Chenglei Si, Davut Emre Taşar, Elizabeth Salesky, Sabrina J. Mielke, Wilson Y. Lee, Abheesht Sharma, Andrea Santilli, Antoine Chaffin, Arnaud Stiegler, Debajyoti Datta, Eliza Szczechla, Gunjan Chhablani, Han Wang, Harshit Pandey, Hendrik Strobelt, Jason Alan Fries, Jos Rozen, Leo Gao, Lintang Sutawika, M Saiful Bari, Maged S.

Al-shaibani, Matteo Manica, Nihal Nayak, Ryan Teehan, Samuel Albanie, Sheng Shen, Srulik Ben-David, Stephen H. Bach, Taewoon Kim, Tali Bers, Thibault Fevry, Trishala Neeraj, Urmish Thakker, Vikas Raunak, Xiangru Tang, Zheng-Xin Yong, Zhiqing Sun, Shaked Brody, Yallow Uri, Hadar Tojarieh, Adam Roberts, Hyung Won Chung, Jaesung Tae, Jason Phang, Ofir Press, Conglong Li, Deepak Narayanan, Hatim Bourfoune, Jared Casper, Jeff Rasley, Max Ryabinin, Mayank Mishra, Minjia Zhang, Mohammad Shoeybi, Myriam Peyrounette, Nicolas Patry, Nouamane Tazi, Omar Sanseviero, Patrick von Platen, Pierre Cornette, Pierre François Lavallée, Rémi Lacroix, Samyam Rajbhandari, Sanchit Gandhi, Shaden Smith, Stéphane Requena, Suraj Patil, Tim Dettmers, Ahmed Baruwa, Amanpreet Singh, Anastasia Cheveleva, Anne-Laure Ligozat, Arjun Subramonian, Aurélie Névéol, Charles Lovering, Dan Garrette, Deepak Tunuguntla, Ehud Reiter, Ekaterina Taktasheva, Ekaterina Voloshina, Eli Bogdanov, Genta Indra Winata, Hailey Schoelkopf, Jan-Christoph Kalo, Jekaterina Novikova, Jessica Zosa Forde, Jordan Clive, Jungo Kasai, Ken Kawamura, Liam Hazan, Marine Carpuat, Miruna Clinciu, Najoung Kim, Newton Cheng, Oleg Serikov, Omer Antverg, Oskar van der Wal, Rui Zhang, Ruochen Zhang, Sebastian Gehrmann, Shachar Mirkin, Shani Pais, Tatiana Shavrina, Thomas Scialom, Tian Yun, Tomasz Limisiewicz, Verena Rieser, Vitaly Protasov, Vladislav Mikhailov, Yada Pruksachatkun, Yonatan Belinkov, Zachary Bamberger, Zdeněk Kasner, Alice Rueda, Amanda Pestana, Amir Feizpour, Ammar Khan, Amy Faranak, Ana Santos, Anthony Hevia, Antigona Unldreaj, Arash Aghagol, Arezoo Abdollahi, Aycha Tammour, Azadeh HajiHosseini, Bahareh Behroozi, Benjamin Ajibade, Bharat Saxena, Carlos Muñoz Ferrandis, Daniel McDuff, Danish Contractor, David Lansky, Davis David, Douwe Kiela, Duong A. Nguyen, Edward Tan, Emi Baylor, Ezinwanne Ozoani, Fatima Mirza, Frankline Ononiwu, Habib Rezanejad, Hessie Jones, Indrani Bhattacharya, Irene Solaiman, Irina Sedenko, Isar Nejadgholi, Jesse Passmore, Josh Seltzer, Julio Bonis Sanz, Livia Dutra, Mairon Samagaio, Maraim Elbadri, Margot Mieskes, Marissa Gerchick, Martha Akinlolu, Michael McKenna, Mike Qiu, Muhammed Ghauri, Mykola Burynok, Nafis Abrar, Nazneen Rajani, Nour Elkott, Nour Fahmy, Olanrewaju Samuel, Ran An, Rasmus Kromann, Ryan Hao, Samira Alizadeh, Sarmad Shubber, Silas Wang, Sourav Roy, Sylvain Viguier, Thanh Le, Tobi Oyebade, Trieu Le, Yoyo Yang, Zach Nguyen, Abhinav Ramesh Kashyap, Alfredo Palasciano, Alison Callahan, Anima Shukla, Antonio Miranda-Escalada, Ayush Singh, Benjamin Beilharz, Bo Wang, Caio Brito, Chenxi Zhou, Chirag Jain, Chuxin Xu, Clémentine Fourrier, Daniel León Periñán, Daniel Molano, Dian Yu, Enrique Manjavacas, Fabio Barth, Florian Fuhrimann, Gabriel Altay, Giyaseddin Bayrak, Gully Burns, Helena U. Vrabec, Imane Bello, Ishani Dash, Jihyun Kang, John Giorgi, Jonas Golde, Jose David Posada, Karthik Rangasai Sivaraman, Lokesh Bulchandani, Lu Liu, Luisa Shinzato, Madeleine Hahn de Bykhovetz, Maiko Takeuchi, Marc Pàmies, Maria A Castillo, Marianna Nezhurina, Mario Sänger, Matthias Samwald, Michael Cullan, Michael Weinberg, Michiel De Wolf, Mina Mihaljcic, Minna Liu, Moritz Freidank, Myungsun Kang, Natasha Seelam, Nathan Dahlberg, Nicholas Michio Broad, Nikolaus Muellner, Pascale Fung, Patrick Haller, Ramya Chandrasekhar, Renata Eisenberg, Robert Martin, Rodrigo Canalli, Rosaline Su, Ruisi Su, Samuel Cahyawijaya, Samuele Garda, Shlok S Deshmukh, Shubhanshu Mishra, Sid Kiblawi, Simon Ott, Sinee Sang-aroonsiri, Srishti Kumar, Stefan Schweter, Sushil Bharati, Tanmay Laud, Théo Gigant, Tomoya Kainuma, Wojciech Kusa, Yanis Labrak, Yash Shailesh Bajaj, Yash Venkatraman, Yifan Xu, Yingxin Xu, Yu Xu, Zhe Tan, Zhongli Xie, Zifan Ye, Mathilde Bras, Younes Belkada, and Thomas Wolf. Bloom: A 176b-parameter open-access multilingual language model, 2023.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer, 2017.

Noam Shazeer, Youlong Cheng, Niki Parmar, Dustin Tran, Ashish Vaswani, Penporn Koanantakool, Peter Hawkins, HyoukJoong Lee, Mingsheng Hong, Cliff Young, Ryan Sepassi, and Blake Hechtman. Mesh-tensorflow: Deep learning for supercomputers, 2018.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. Llama: Open and efficient foundation language models, 2023a.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn,

Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. Llama 2: Open foundation and fine-tuned chat models, 2023b.

Shukang Yin, Chaoyou Fu, Sirui Zhao, Ke Li, Xing Sun, Tong Xu, and Enhong Chen. A survey on multimodal large language models, 2023.

Yanqi Zhou, Tao Lei, Hanxiao Liu, Nan Du, Yanping Huang, Vincent Zhao, Andrew Dai, Zhifeng Chen, Quoc Le, and James Laudon. Mixture-of-experts with expert choice routing, 2022.

Yanqi Zhou, Nan Du, Yanping Huang, Daiyi Peng, Chang Lan, Da Huang, Siamak Shakeri, David So, Andrew Dai, Yifeng Lu, Zhifeng Chen, Quoc Le, Claire Cui, James Laundon, and Jeff Dean. Brainformers: Trading simplicity for efficiency, 2023.

A ARCHITECTURE AND TRAINING SETUP

All of the models considered in this work are decoder-only Transformers trained on the C4 dataset (Raffel et al., 2023). We use GPT2 tokenizer (Radford et al., 2018a). Each batch consists of 0.5M tokens packed into 2048 sequences. Our optimizer is AdamW (Loshchilov & Hutter, 2019), with a weight decay of 0.1. In each training run, we use the maximum learning rate of $2\mathrm{e}{-4}$, with linear warmup for 1% steps and cosine decay to $2\mathrm{e}{-5}$. To improve stability, we initialize weights using the truncated normal distribution with reduced scale, as advised in Fedus et al. (2022). The models are trained using mixed precision; we always keep the attention mechanism and router in high precision. We assume the *infinite data* regime, as the number of training tokens for any of the runs is less than the number of tokens in the corpus. We follow Hoffmann et al. (2022) and perform our analysis on the smoothed training loss.

In MoE, we use the Expert Choice routing algorithm, as it guarantees a balanced expert load without tuning additional hyperparameters. To maintain compatibility with autoregressive language modeling, we apply the recipe described in Zhou et al. (2022): tokens are grouped by position across different sequences. The group size is always set to 256. We match the number of FLOPs for MoE and dense models with the same d_{model} (meaning we activate an average of $8d_{\text{model}}^2$ parameters per token in each MoE layer). In the router, softmax is performed over the expert dimension, while we choose tokens over the token dimension, as this leads to the best performance (as opposed to performing softmax over the token dimension). We put an additional layer normalization before the output of MoE layer. This gives a small improvement for standard MoE, but is crucial for the performance of models with $G > 1$.

Table 3 and Table 4 list the considered architecture and training variants for dense and MoE models, respectively.

Table 3: Architecture and training variants (MoE models).

#parameters (nonemb)	d_{model}	n_{blocks}	n_{heads}	D (in #tokens)	G
64x3M	256	4	4	16B, 33B, 66B	1, 2, 4, 8, 16
64x7M	384	4	6	16B, 33B, 66B	1, 2, 4, 8, 16
64x13M	512	4	8	16B, 33B, 66B	1, 2, 4, 8, 16
64x13M	512	4	8	130B	1, 2, 4
64x25M	512	8	8	16B, 33B,	1, 2, 4, 8, 16
64x25M	512	8	8	66B	1, 2, 4, 8
64x49M	640	10	10	16B, 33B	1, 2, 4, 8, 16
64x49M	640	10	10	66B	1, 2, 4
64x85M	768	12	12	33B	1, 2, 4

Table 4: Architecture and training variants (dense models).

#parameters (nonemb)	d_{model}	n_{blocks}	n_{heads}	D (in #tokens)
3M	256	4	4	16B, 24B, 33B, 66B
6M	256	8	4	16B, 24B, 33B, 66B
13M	512	4	8	16B, 24B, 33B, 66B
25M	512	8	8	16B, 24B, 33B, 66B
49M	640	10	10	16B, 24B, 33B, 66B
85M	768	12	12	16B, 33B

B Validation of the Scaling Law

In this section, we provide coefficients of the scaling law fitted with 20% of datapoints with the lowest perplexity excluded for the purpose of validation.

Table 5: Values of the fitted coefficients.

Model	a	α	b	β	g	γ	c
MoE	17.6	0.114	26.7	0.140	2.07	0.570	0.472

C Reliability of Compute Optimal Formula

In this section, we assess the stability of our predictions presented in Section 6.1. Similarly to Hoffmann et al. (2022) we calculate the 10th and 90th percentiles estimated via bootstrapping data (80% of the data is sampled 100 times). See Table 6 for the details.

Table 6: 10th and 90th percentiles estimated via bootstraping data.

N	D	G
64 x 100M	(2.97B, 5.98B)	(8, 8)
64 x 1B	(21.17B, 40.73B)	(16, 16)
64 x 3B	(50.20B, 105.88B)	(16, 32)
64 x 7B	(101.06B, 205.40B)	(32, 32)
64 x 70B	(638.49B, 1.59T)	(32, 64)
64 x 300B	(1.99T, 5.62T)	(64, 64)
64 x 1T	(5.29T, 16.87T)	(64, 64)

D Varying Expansion Rate

In this section, we provide results for $E = 16$. The training procedure is the same as described in App. A. The models considered in this part are listed in Table 7.

Table 7: Architecture and training variants (MoE models).

#parameters (nonemb)	d_{model}	n_{blocks}	n_{heads}	D (in #tokens)	G
64x3M	256	4	4	8B, 16B, 33B	1, 2, 4, 8, 16
64x7M	256	8	4	8B, 16B, 33B	1, 2, 4, 8, 16
64x13M	512	4	8	8B, 16B, 33B	1, 2, 4, 8, 16
64x13M	512	4	8	66B	1, 2, 4
64x25M	512	8	8	8B, 16B, 33B	1, 2, 4, 8, 16
64x49M	640	10	10	8B	1, 2, 4, 8, 16

We fit Eq. 9 using the same procedure as described in Section 5.4. The results are detailed in Table 8.

Table 8: Values of the fitted coefficients.

Model	a	α	b	β	g	γ	c
MoE ($E = 16$)	19.64	0.124	57.07	0.169	1.18	0.986	0.472

Using the coefficients and FLOPs calculation formulas, we can derive the compute optimal training parameters. The results are presented in Table 9.

Table 9: 10th and 90th percentiles estimated via bootstrapping data for $E = 16$.

N	D	G
16 x 100M	(10.29B, 17.73B)	(8 , 16)
16 x 1B	(53.74B, 103.54B)	(16, 32)
16 x 3B	(106.22B, 261.04B)	(16, 32)
16 x 7B	(177.65B, 511.43B)	(16, 32)
16 x 70B	(721.60B, 3.22T)	(32, 64)
16 x 300B	(1.73T, 10.69T)	(32, 64)
16 x 1T	(3.60T, 28.22T)	(32, 128)

We can observe that similarly to the case when $E = 64$, larger compute budgets imply larger optimal values of G. Note that the values for 10th and 90th percentiles form larger intervals in this case, as in this part we run a smaller number of experiments and keep shorter training durations. However, we believe that this preliminary study forms a valuable addition to the results in the main part.

E FLOPs Constants

The number of FLOPs F used in Transformer training, considering the routing operation overhead in MoE, can be described by the following formula:

$$F = (12{d_{\text{model}}}^2 c_f + d_{\text{model}} E G c_r) \cdot n_{\text{tokens}} \cdot n_{\text{layers}} \tag{11}$$

Following Hoffmann et al. (2022), we assume c_f to be 6. This is interpreted as 6 FLOPs for each pair of an active parameter (in linear projection) and a processed token. The breakdown of operations is as follows:

- During the forward pass, 2 operations (single multiplication and single addition) are used to compute the matrix multiplication of an input and linear projection.
- During the backward pass, 2 operations are used to compute gradients wrt. the input.
- During the backward pass, 2 operations are used to compute gradients wrt. the weights of linear projection.

In our work, we have assumed the routing constant, c_r, to be 14, with the breakdown presented below. The exact number of operations may depend on the implementation of routing, but it will be between 6 and 20. However, our main conclusions of the paper are resistant to different assumptions of this constant.

- During the forward pass, 2 operations are used to compute the expert logits based on an input and "routing linear projection".
- During the backward pass, 2 operations are used to compute gradients for "routing linear projection" wrt. the input.
- During the backward pass, 2 operations are used to compute gradients for "routing linear projection" wrt. the weights of linear projection.
- During the forward pass, 2 operations are used to route input tokens to chosen experts.
- During the forward pass, 2 operations are used to route expert outputs to chosen tokens and multiply those outputs by the routing score.
- During the backward pass, 2 operations are used to route gradients from output tokens to experts.
- During the backward pass, 2 operations are used to route gradients from experts to input tokens.

Similarly to the calculation of FLOPs for c_f, FLOPs come in pairs as each multiplication is followed by an addition (used to accumulate outputs or gradients).

F Additional Visualizations

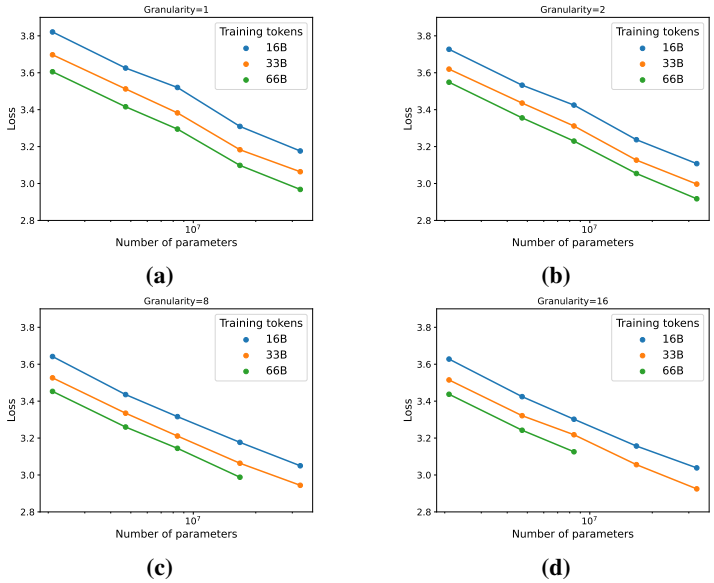

Figure 6: Illustration of scaling N and D for constant granularity value of: **(a)** $G = 1$ **(b)** $G = 2$ **(c)** $G = 8$ **(d)** $G = 16$.

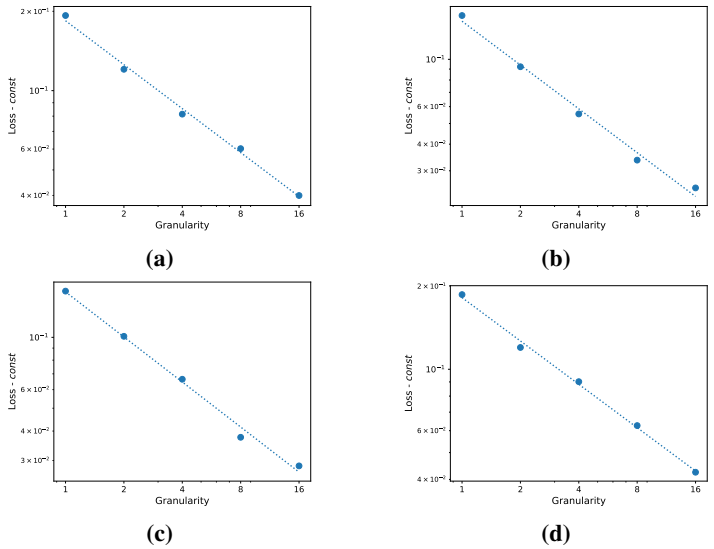

Figure 7: Illustration of scaling granularity when N, D are fixed for: **(a)** $N = 64 \times 25M$, $D = 16B$, $const = 3.12$ **(b)** $N = 64 \times 49M$, $D = 16B$, $const = 3.02$ **(c)** $N = 64 \times 25M$, $D = 32B$, $const = 3.03$ **(d)** $N = 64 \times 49M$, $D = 32B$, $const = 2.88$